

HTTP Status Codes

How to interpret HTTP responses in API testing.

Overview

HTTP codes are grouped into 1xx informational, 2xx success, 3xx redirection, 4xx client error and 5xx server error.

Official site and documentation

Use the official HTTP Status Codes source as the primary reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Essential QA codes

Validate 200, 201 and 204 for success; 400, 401, 403, 404, 409, 422 and 429 for client scenarios; and 500, 502, 503 and 504 for server/dependency failures.

Test strategy

Do not validate the number alone. Check body, headers, contract, business rules, idempotency and sensitive-data exposure.

When to use it

Use this knowledge when product risk, architecture or the delivery stage requires this type of validation. Confirm the goal, environment, required data and expected evidence before starting.

How QA applies it

Start from the behavior or risk to validate, prepare data and environment, execute the technique or tool reproducibly and capture evidence that helps the team understand the result.

Good practices

Prioritize high-impact scenarios, keep data and environments controlled, record important decisions and review the approach periodically to avoid redundant or outdated coverage.

What it is and where it fits

HTTP Status Codes is a tool or technology that can support Quality Engineering work. Before adopting it, understand the problem it solves, where it fits in the workflow and what evidence or productivity gain it should provide.

First QA-oriented use

Pick one small, verifiable task in HTTP Status Codes, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where HTTP Status Codes fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether HTTP Status Codes still delivers value as the product evolves.

Common issues and troubleshooting

When HTTP Status Codes fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.